

COMPLETE CODE EXPLANATION & INTERVIEW PREPARATION GUIDE

BrainBarter

Every file. Every function. Every line. Everything you need to explain this project like you coded it from scratch.

RAG Pipeline

pgvector

HMAC Payments

RLS Security

React 19

Node.js

Express

Supabase

PostgreSQL

Google Gemini

Razorpay

SendGrid

Zustand

Vercel · Render

Project

BrainBarter — P2P Learning Marketplace

Developer

Abhay Kumar Dwivedi

Live

brain-barter-zeta.vercel.app

Sections

22 sections · 40 Interview Q&As



Table of Contents

1. Concepts You Must Know (REST, JWT, CORS, HMAC, RAG, RLS, Zustand...)
2. SERVER: index.js — Entry point, Express setup, CORS, routing
3. SERVER: verifyToken.js — JWT middleware, service key
4. SERVER: authController.js — OTP generation, SendGrid, Supabase upsert
5. SERVER: paymentController.js — Razorpay, HMAC, idempotent credit
6. SERVER: gemini.js — System prompts, callGemini, model selection
7. SERVER: embeddings.js — embedText, chunkText, overlap logic
8. SERVER: aiController.js — RAG ingestion, retrieval, examMode
9. CLIENT: supabase.js — createClient, anon key, env vars
10. CLIENT: authStore.js — Zustand store, global state
11. CLIENT: themeStore.js — persist middleware, dark mode
12. CLIENT: ProtectedRoute.jsx — auth guard, admin guard, redirect
13. CLIENT: App.jsx — BrowserRouter, Routes, session restore
14. CLIENT: Upload.jsx — file upload flow, autocomplete, RAG ingest trigger
15. CLIENT: ContentPage.jsx — unlock, rating, AI panel, citations
16. CLIENT: ExamMode.jsx — 5 tabs, study planner, AI exam API
17. CLIENT: Wallet.jsx — buy tokens, Razorpay checkout, withdrawal
18. CLIENT: FeedbackButton.jsx — floating modal, anonymous insert
19. Database (COMPLETE_SETUP.sql) — RLS, SECURITY DEFINER, pgvector
20. 40 Interview Questions & Answers
21. "Explain the project in 2 minutes" — memorisable script

Concept	What it is	Why we used it	Alternatives & why not
REST API	A style of building web APIs where each URL represents a resource and HTTP methods (GET/POST/PUT/DELETE) define actions. Stateless — each request carries all needed info.	Our Express server exposes REST endpoints like POST /api/payment/verify. Simple, well-understood, works with any frontend.	GraphQL (overkill for our use case), gRPC (needs special clients). REST was simplest.
Node.js	JavaScript runtime that lets you run JS on the server (outside the browser). Uses V8 engine. Single-threaded but non-blocking via event loop.	Same language as frontend (React), huge ecosystem (npm), excellent for I/O-heavy tasks like calling APIs.	Python/Django, Java/Spring — different language adds complexity. PHP — outdated ecosystem.
Express	Minimal web framework for Node.js. Adds clean routing (app.get, app.post), middleware support, error handling on top of raw Node http module.	Raw Node.js HTTP is verbose. Express makes routes readable in 2 lines.	Fastify (faster but less docs), Koa (smaller community), Hapi (opinionated). Express has most tutorials/help.
Middleware	Function that runs between a request arriving and the handler running. Has access to req, res, and next(). Calling next() passes control forward.	Our verifyToken middleware authenticates every protected route in one place — no repeated auth code in each controller.	Could inline auth in every handler — but that's copy-paste and error-prone.
JWT	JSON Web Token — a cryptographically signed string containing user info (id, email). Format: header.payload.signature. Cannot be faked without the secret key.	Supabase issues JWTs on login. Our server validates them to know who is making each request. Stateless — no session DB needed.	Session cookies (need a session store), API keys (don't expire automatically).
CORS	Cross-Origin Resource Sharing. Browser security policy that blocks JS from calling a different domain unless the server explicitly allows it via response headers.	Our React app (vercel.app) calls our API (render.com) — different domains. Without CORS config, browsers block every request.	No alternative — CORS is enforced by browsers, you must configure it.

Concept	What it is	Why we used it	Alternatives & why not
HMAC	Hash-based Message Authentication Code. Takes data + secret key → produces a fixed-length hash. If data or key changes, hash changes completely.	We use HMAC-SHA256 to verify Razorpay payment signatures. Proves the payment notification came from Razorpay and wasn't tampered with.	No real alternative for this use case — Razorpay requires HMAC verification.
Idempotency	An operation that produces the same result no matter how many times you call it. $f(f(x)) = f(x)$. Critical for payment systems where retries happen.	Our credit_purchase() DB function checks UNIQUE(payment_id) so a repeated Razorpay webhook never double-credits tokens.	Without it: if Razorpay retries (it does), users get tokens twice for one payment.
RAG	Retrieval-Augmented Generation. Instead of answering from training data, the AI retrieves relevant documents first, then generates an answer grounded in those docs.	Standard Gemini answers from general knowledge. We want it to answer from the student's actual uploaded PDF. RAG makes that possible.	Fine-tuning (expensive, needs lots of data). Prompt stuffing entire PDF (too large). RAG is the right middle ground.
Embeddings	Converting text to a list of numbers (a vector) that captures meaning. Similar text → similar numbers. Generated by an ML model.	We embed each PDF chunk and each question using Gemini's text-embedding-004. Then find closest chunks to the question using cosine similarity.	OpenAI embeddings (paid, different vendor), sentence-transformers (self-hosted, complex). Gemini embeddings are free with our existing API key.
pgvector	PostgreSQL extension that adds a vector data type and similarity search operators. Lets you store 768 numbers per row and find nearest neighbours efficiently.	We store Gemini embeddings in PostgreSQL and query them with cosine similarity to find relevant PDF chunks.	Pinecone/ Weaviate (separate vector DB services, extra cost, extra infrastructure). pgvector keeps everything in one DB.
Cosine Similarity	Measures the angle between two vectors. 1 = same direction (identical meaning), 0 = perpendicular (unrelated), -1 = opposite. Better than	We find PDF chunks most semantically similar to a student's question even if they use different words.	Dot product (doesn't normalise for length), Euclidean distance (less

Concept	What it is	Why we used it	Alternatives & why not
	keyword search because it understands semantic meaning.		suited for text). Cosine is standard for text similarity.
Row-Level Security	PostgreSQL feature where each table has SQL policies controlling which rows each user can SELECT/INSERT/UPDATE/DELETE. Enforced at DB level regardless of how data is accessed.	Clients access Supabase directly. Without RLS, any user could UPDATE their own token_balance. RLS blocks this at DB level.	Application-level checks only (can be bypassed if you hit DB directly). RLS cannot be bypassed — it's in the DB engine.
SECURITY DEFINER	A PostgreSQL function attribute meaning "run this function with the privileges of the function's owner, not the caller." Allows bypassing RLS safely for specific audited operations.	Our credit_purchase() function needs to update token_balance (blocked by RLS for users). SECURITY DEFINER lets it do so safely — only this specific function can, not arbitrary SQL.	Could give users elevated DB permissions — but that defeats the security model.
Zustand	Tiny React state management library. Creates a global store any component can subscribe to. No boilerplate — just define state and setters.	User auth state (user, profile) needs to be accessible from Navbar, ContentPage, Wallet, etc. Zustand shares it without prop drilling.	Redux (too much boilerplate), React Context (re-renders all consumers on any change). Zustand is minimal and performant.
Supabase	Open-source Firebase alternative. Provides PostgreSQL database, Auth (JWT), Storage (S3-compatible), and auto-generated APIs — all in one.	Gets us database + auth + file storage without managing servers. Supports RLS and pgvector natively.	Firebase (NoSQL, no SQL queries, no pgvector), PlanetScale (MySQL, no pgvector), self-hosted PostgreSQL (complex ops).


```
require('dotenv').config()
```

What it does: Loads the `.env` file and puts every key=value pair into `process.env`. So after this line, `process.env.GEMINI_API_KEY` exists. Without this, all env vars would be `undefined` and every API call would fail. **Why first line?** Other modules read `process.env` when they load — if dotenv runs after them, they'd see undefined.

```
const express = require('express')
const app = express()
```

What it does: Imports Express and creates an application instance. `app` is the object you attach routes and middleware to. Think of it as the blank restaurant — nothing in it yet, just the building.

```
app.use(cors({
  origin: (origin, callback) => {
    const allowedOrigins = [
      process.env.CLIENT_URL,
      'https://brain-barter-zeta.vercel.app',
      /\.vercel\.app$/
    ]
    if (!origin || allowedOrigins.some(o => o instanceof RegExp ? o.test(origin) : o === origin)) {
      callback(null, true)
    } else {
      callback(new Error('Not allowed by CORS'))
    }
  }
}))
```

What CORS does: Browsers block JavaScript from calling a different domain (e.g. your React app on vercel.app calling your API on render.com) unless the server says "I allow this origin." The `cors()` middleware adds the right response headers automatically.

Why a function instead of a string? We have multiple allowed origins — the deployed URL, the environment variable (for local dev), and a RegExp pattern that matches ANY Vercel preview URL (e.g. brainbarter-abc123.vercel.app). A function lets us check all three.

`!origin` means "no origin header" — this happens for server-to-server requests (Postman, curl). We allow those too.

`callback(null, true)` = allowed. `callback(new Error(...))` = blocked.

```
app.use(express.json())
```

HTTP request bodies arrive as raw bytes. This middleware parses JSON bodies automatically so you can write `req.body.email` instead of manually parsing. Without this, `req.body` would be undefined for all POST requests.

```
app.get('/health', (_, res) => res.json({ status: 'ok' })))
```

A health check endpoint. Render and uptime monitoring services ping this URL periodically. If they get a 200 response, the server is alive. `_` instead of `req` is a convention meaning "I don't use this parameter."

```
app.use('/api/ai', aiRoutes)
app.use('/api/auth', authRoutes)
app.use('/api/payment', paymentRoutes)
```

Route mounting. Any request starting with `/api/ai` is forwarded to the `aiRoutes` file to handle. This is separation of concerns — all AI logic in one file, all payment logic in another. Makes the codebase maintainable.

```
app.use((err, req, res, next) => {
  console.error(err.stack)
  res.status(500).json({ error: 'Internal server error' })
})
```

Global error handler. Express recognises a middleware with 4 parameters as an error handler. If any route throws an uncaught error, Express passes it here instead of crashing. We log the stack trace (for debugging) and return a generic 500 response (never expose internal errors to clients).

```
const PORT = process.env.PORT || 5000
app.listen(PORT, () => console.log(`Server running on port ${PORT}`))
```

`process.env.PORT` is set by Render (the hosting platform) at runtime. The `|| 5000` fallback is for local development where PORT isn't set. `app.listen()` starts the HTTP server — after this line, the server accepts incoming requests.

Interview Questions — index.js

Q: Why did you put `dotenv.config()` as the very first line?

A: Other modules read `process.env` when they're first imported. If `dotenv` ran after those imports, they'd see undefined for all environment variables. It must run before any other `require()` statement.

Q: What is CORS and why did you need to configure it?

A: CORS is a browser security feature that blocks JavaScript from calling a different domain than the one serving the page. My React app runs on `vercel.app` but the API is on `render.com` — different domains. Without CORS configuration, every API call from the browser would be blocked. The server adds response headers like `Access-Control-Allow-Origin` to tell the browser which origins are permitted.

Q: What does the global error handler do and why is it at the bottom?

A: It catches any unhandled errors thrown in route handlers and returns a clean 500 response instead of crashing the server. It must be defined LAST because Express processes middleware in

order — if it were first, it would never see errors thrown by routes registered after it. The 4-parameter signature (err, req, res, next) is how Express identifies it as an error handler.

What is a Middleware?

Middleware is a function that sits between the HTTP request arriving and your actual route handler running. It has three jobs: inspect/modify the request, inspect/modify the response, or call `next()` to pass control to the next middleware/handler. If you DON'T call `next()`, the request just hangs — the client gets no response.

```
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERVICE_KEY)
```

Service key vs Anon key: There are two Supabase keys. The **anon key** (used in React) is public — it's safe to expose in frontend code because Row-Level Security controls what it can do. The **service key** has full database access — it bypasses RLS entirely. It MUST stay on the server only. We use the service key here because this middleware needs to validate any JWT, which requires elevated access.

```
async function verifyToken(req, res, next) {
  const auth = req.headers.authorization
  if (!auth || !auth.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' })
  }
  const token = auth.split(' ')[1]
  const { data: { user }, error } = await supabase.auth.getUser(token)
  if (error || !user) return res.status(401).json({ error: 'Invalid token' })
  req.user = user
  next()
}
```

`req.headers.authorization` — HTTP headers are key-value pairs sent with every request. The `Authorization` header follows the format `Bearer <token>` by convention.

`auth.split(' ')[1]` — "Bearer eyJhbGci..." split by space gives ["Bearer", "eyJhbGci..."]. Index [1] extracts just the JWT string.

`supabase.auth.getUser(token)` — sends the JWT to Supabase's auth server, which verifies the cryptographic signature and returns the user object if valid. This is the source of truth — you can't fake a JWT without Supabase's secret signing key.

`req.user = user` — attaches the verified user object to the request. Every downstream handler can now trust `req.user.id` — it's the real authenticated user's ID.

`next()` — tells Express "authentication passed, proceed to the actual route handler." Without this call, the request would hang forever.

Interview Questions — verifyToken**Q: What is a JWT and how does your server verify it?**

A: JWT stands for JSON Web Token. It's a base64-encoded string with three parts separated by dots: a header (algorithm info), a payload (user data like id and email), and a signature. The signature is

created by hashing the header+payload with a secret key. When our server receives a JWT, we call `supabase.auth.getUser(token)` which sends it to Supabase's auth server — they verify the signature using their secret key and return the user if it's valid. A forged or expired JWT fails verification and we return 401.

Q: Why is the service key used here instead of the anon key?

A: The anon key respects Row-Level Security — it can only do what the RLS policies allow for unauthenticated users. Verifying a JWT requires calling `supabase.auth.getUser()` which needs elevated privileges. The service key bypasses RLS and has full access. It's only used on the server (never exposed to the browser) precisely because it's so powerful.

Q: What happens if someone sends a request without a token?

A: The middleware checks if the Authorization header exists and starts with "Bearer ". If not, it immediately returns a 401 Unauthorized response and does NOT call `next()` — so the actual route handler never runs. The request is dead at the middleware stage.


```
const code = (crypto.randomInt(0, 1000000)).toString().padStart(6, '0')
```

`crypto.randomInt()` uses the OS's cryptographically secure random number generator (CSPRNG). It is truly unpredictable. **Why not Math.random()? Math.random()** is a pseudo-random number generator — its output can theoretically be predicted if you know the seed. For security codes, predictability is dangerous. `crypto` is Node's built-in module — no install needed.

`.padStart(6, '0')` — if `randomInt` gives 42, `toString` gives "42", `padStart` makes it "000042". Ensures the code is always exactly 6 characters.

```
const expires = new Date(Date.now() + 10 * 60 * 1000).toISOString()
```

`Date.now()` = current time in milliseconds. `10 * 60 * 1000` = 10 minutes in milliseconds. Adding them gives the expiry timestamp. `.toISOString()` converts to "2024-01-01T12:10:00.000Z" format — what PostgreSQL expects for timestamps.

```
await supabase.from('email_verification_codes')
  .upsert({ email, code, expires_at: expires }, { onConflict: 'email' })
```

`upsert` = INSERT if email doesn't exist, UPDATE if it does. This handles "resend code" gracefully — replaces the old code with a fresh one instead of inserting a duplicate. `onConflict: 'email'` specifies which column to check for conflict.

Why Supabase table instead of a variable? Our server runs on Render's free tier which sleeps after inactivity. When it wakes up, all in-memory variables are gone. Storing codes in the database survives restarts. Also critical for multi-instance deployments.

```
// verifyCode: check expiry, compare, delete on success
if (new Date() > new Date(data.expires_at)) {
  await supabase.from('email_verification_codes').delete().eq('email', email)
  return res.status(400).json({ error: 'Code expired' })
}
if (data.code !== code) return res.status(400).json({ error: 'Invalid code' })
await supabase.from('email_verification_codes').delete().eq('email', email)
```

Expiry check: Compare current time with stored expiry. If expired, delete the code (clean up) and reject.

Code check: Simple string equality. **Delete on success:** OTPs are single-use. After verification, immediately delete so the same code can't be used again (prevents replay attacks).

Interview Questions — authController

Q: Why did you use `crypto.randomInt` instead of `Math.random` for generating OTP codes?

A: `Math.random()` is a pseudo-random generator — it uses a deterministic algorithm seeded by a value. In theory, if an attacker knows the seed or observes enough outputs, they can predict future values. For security-critical codes like OTPs, this is unacceptable. `crypto.randomInt()` uses the OS's

cryptographically secure random number generator (like `/dev/urandom` on Linux), which is truly unpredictable. Node's built-in crypto module is always available — no extra dependencies.

Q: Why store verification codes in the database instead of a JavaScript variable or Map?

A: Our backend runs on Render's free tier which sleeps after 15 minutes of inactivity. When it wakes up, all JavaScript memory is reset — any in-memory Map would be empty. Users who requested a code before the sleep would find their code gone. By storing codes in Supabase, they persist across restarts. This is also necessary for horizontal scaling — if you run multiple server instances, an in-memory Map on instance 1 is invisible to instance 2.

Q: Why delete the code after successful verification?

A: OTPs must be single-use. If we didn't delete after verification, an attacker who observed the code (e.g., shoulder surfing) could verify again later. Deleting immediately after use prevents replay attacks. Expired codes are also deleted when detected to keep the table clean.

Why Two Steps? (createOrder → verifyPayment)

If payment and token-crediting happened in one client-side step, a user could modify the JavaScript to say "charge ₹1 but credit 10,000 tokens." By splitting: (1) server creates order and locks the token amount in order.notes, (2) Razorpay handles the actual payment on their secure servers, (3) server verifies the payment is real using HMAC before crediting exactly what the order specified — the client can never alter the token amount.

```
const order = await razorpay.orders.create({
  amount: inr * 100, // Razorpay uses paise (₹1 = 100 paise)
  currency: 'INR',
  receipt: `bb_${Date.now()}`, // unique receipt ID
  notes: { user_id: req.user.id, tokens: String(tokens), inr: String(inr) },
})
```

`notes` is a key-value store attached to the Razorpay order. We store how many tokens this payment buys here on the server. When the payment completes, we fetch the order and read `notes.tokens` — the client cannot modify this value.

```
const expected = crypto
  .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`)
  .digest('hex')

if (expected !== razorpay_signature) {
  return res.status(400).json({ error: 'Invalid payment signature' })
}
```

HMAC step by step:

- ▶ `createHmac('sha256', secret)` — creates an HMAC generator using SHA256 hashing algorithm and your Razorpay secret as the key
- ▶ `.update(`order_id|payment_id`)` — feeds the data to hash. The pipe character `|` is Razorpay's separator
- ▶ `.digest('hex')` — finalises and returns the hash as a hex string
- ▶ Razorpay does the same computation on their end and sends the result as `razorpay_signature`
- ▶ If they match → payment is authentic. If not → someone tampered with the data

```
const { data: balance, error } = await supabase.rpc('credit_purchase', {
  p_user_id: req.user.id, p_tokens: tokens, p_payment_id: razorpay_payment_id,
})
```

Why an RPC instead of a direct UPDATE? `credit_purchase()` is a SECURITY DEFINER PostgreSQL function that: (1) checks if `payment_id` already exists (idempotent — prevents double-credit), (2) adds

tokens to the user's balance, (3) inserts a transaction record. All three steps in one atomic database transaction — either all succeed or all roll back. A direct UPDATE would require multiple queries and could leave the DB in a broken state if the server crashed between them.

Interview Questions — Payments

Q: Explain the full Razorpay payment flow in your app, end to end.

A: (1) User clicks "Buy Tokens" → client sends amount to our POST /api/payment/create-order. (2) Server creates a Razorpay order, storing how many tokens this buys in order.notes, and returns the orderId. (3) Client opens the Razorpay checkout popup using the orderId. User enters card/UPI details — this happens entirely on Razorpay's servers. (4) On success, Razorpay gives the client three values: order_id, payment_id, and a signature. (5) Client sends all three to our POST /api/payment/verify. (6) Server recomputes the HMAC signature using our secret key. If it matches Razorpay's signature, the payment is authentic. (7) Server calls credit_purchase() RPC to atomically add tokens.

Q: What is HMAC and why is it used for payment verification?

A: HMAC stands for Hash-based Message Authentication Code. You combine a message with a secret key and run it through a hash function (SHA256 in our case). The result is a fixed-length string that uniquely represents that message+key combination. Razorpay computes HMAC(order_id + payment_id, your_secret) and sends it as the signature. Our server does the same computation. If they match, we know: (1) the message wasn't tampered with — any change would produce a different hash, and (2) it came from Razorpay — only they and we know the secret key.

Q: What prevents a user from paying once and getting tokens twice?

A: Our credit_purchase() database function has a UNIQUE constraint on the payment_id column in the purchases table. The first call inserts the payment record and credits tokens. If the same payment_id arrives again (Razorpay retries, user double-submits, network glitch), the INSERT hits the unique constraint and the function returns without crediting again. This property — same input, same output, no matter how many times called — is called idempotency.

What is a System Prompt?

When calling an LLM (Large Language Model), you provide two things: a system prompt (instructions telling the AI what role to play and how to behave) and the actual content/question. The system prompt is invisible to the end user. Ours say things like "You are an academic assistant. Summarize in 5-7 bullet points." This constrains the AI to give structured, relevant responses instead of generic conversation.

```
const systemPrompts = {
  summarize: 'Summarize in 5-7 bullet points...',
  doubt: 'Content below is numbered passages [1],[2]... Answer using ONLY these. Cite inline like [1].',
  'mock-test': 'Create a 10-question MCQ test with 4 options each...',
  'study-plan': 'Create detailed day-by-day study plan. Day N: Goal, subtopics, activities, self-check...',
}
```

Each key maps a task name to a system prompt string. The **doubt prompt** is special — it explicitly tells Gemini to answer ONLY from the numbered passages we provide (the retrieved RAG chunks) and to cite which passage it used. This is what grounds answers in the actual PDF content.

The **study-plan prompt** is what makes the Study Planner "agentic" — it asks Gemini to plan across multiple days with progression, which is fundamentally different from answering a single question.

```
const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' })
const prompt = `${systemPrompt}\n\nContent:\n${contentText}${userMessage ? '\n\nStudent question: ${userMessage}\n' : ''}`
const result = await model.generateContent(prompt)
return result.response.text()
```

gemini-1.5-flash is Google's fast, low-cost generative model. "Flash" variants are optimised for speed over capability — suitable for our summarisation, Q&A, and test generation tasks that don't need the most powerful model.

The prompt is constructed by concatenating: system instruction + content text + optional student question. The `\n\n` separators help the model understand which part is which.

Interview Questions — AI/Gemini

Q: How do you make the AI answer from a student's PDF instead of its training data?

A: Through a technique called RAG — Retrieval-Augmented Generation. We don't send the whole PDF to Gemini (it could be too large). Instead, when a question is asked, we find the most relevant passages from the PDF using vector similarity search (pgvector), then inject those passages into the prompt as numbered items. The system prompt explicitly says "Answer using ONLY these passages and cite which ones you used." Gemini follows the instruction because system prompts constrain its behaviour.

Q: What is the difference between the tasks summarize and doubt in your AI system?

A: For "summarize", we feed the full indexed text of the document (all chunks in order) and ask Gemini to condense it. For "doubt", we first embed the student's specific question, find the 6 most semantically relevant chunks using pgvector's cosine similarity search, then send only those chunks. The doubt task uses RAG retrieval; summarize uses the full text. The doubt system prompt also instructs Gemini to cite which passage it used, producing verifiable answers.

What is a Vector Embedding?

An embedding converts text into a list of numbers (a vector) that captures the semantic meaning. "SQL joins combine rows" and "how to merge two database tables" would produce very similar vectors even though they share no words — because they mean the same thing. Gemini's text-embedding-004 produces 768 numbers per text input. These numbers are meaningless individually but their relationships encode meaning. Cosine similarity between two vectors measures how similar their meanings are: 1.0 = identical meaning, 0.0 = unrelated.

```
async function embedText(text) {
  const model = genAI.getGenerativeModel({ model: 'text-embedding-004' })
  const result = await model.embedContent(text)
  return result.embedding.values // array of 768 floats
}
```

`text-embedding-004` is Gemini's embedding model — separate from the generative model. It takes text and returns an array of 768 floating-point numbers. We use the same model for both indexing (PDF chunks) and querying (student questions) — this is critical. If you used different models, the vector spaces wouldn't be comparable.

```
function chunkText(text, size = 220, overlap = 40) {
  const words = text.replace(/\s+/g, ' ').trim().split(' ')
  const chunks = []
  for (let i = 0; i < words.length; i += size - overlap) {
    const slice = words.slice(i, i + size).join(' ').trim()
    if (slice) chunks.push(slice)
    if (i + size >= words.length) break
  }
  return chunks
}
```

Why chunk? A 50-page PDF might have 25,000 words. Embedding 25,000 words at once would exceed token limits and lose precision. We split into 220-word pieces.

Why 40-word overlap? If chunk 1 is words 1-220 and chunk 2 is words 181-400, words 181-220 appear in both. This means a concept that spans the boundary of two chunks is fully captured in at least one chunk. Without overlap, a key sentence split across a boundary would be cut in half in both chunks — potentially making neither retrievable.

`text.replace(/\s+/g, ' ')` — replaces all whitespace sequences (multiple spaces, tabs, newlines) with a single space. Normalises text from PDFs which often have irregular spacing.

`i += size - overlap` — step forward by (220-40=180) words each iteration. So chunk 1 starts at 0, chunk 2 at 180, chunk 3 at 360, etc.

Interview Questions — Embeddings

Q: Explain what vector embeddings are and why you use them.

A: An embedding converts text into an array of numbers that represents its meaning mathematically. I use Gemini's text-embedding-004 model which produces 768 numbers per input. The key insight is that semantically similar text produces numerically similar vectors. So "normalisation reduces redundancy" and "why is database normalisation important" would produce vectors that are close together in 768-dimensional space, even though they share no words. This lets us find relevant PDF passages for a student's question even when the question uses different words than the document.

Q: Why did you use 40-word overlap in your text chunking?

A: To avoid information loss at chunk boundaries. Imagine a key sentence: "Therefore, the third normal form requires..." — if "Therefore, the third normal form" ends chunk 1 and "requires..." starts chunk 2, both chunks are incomplete and might not be retrieved when someone asks about 3NF. With 40-word overlap, those 40 words appear in both chunk 1 and chunk 2, so the complete sentence exists in at least one chunk. It trades a small amount of storage for significantly better retrieval quality.

Function 1: ingestContent — Index a PDF for AI

- ▶ **Guard:** `content.creator_id !== req.user.id` — only the creator can trigger indexing of their content. Prevents expensive embedding operations being run on someone else's files.
- ▶ **PDF extraction:** Download file from Supabase Storage URL, pass buffer to pdf-parse library which extracts plain text from the PDF binary format.
- ▶ **Idempotent:** `DELETE content_chunks WHERE content_id = X` before inserting. If re-uploaded or re-indexed, old chunks are cleanly replaced. No duplicates.
- ▶ **Sequential embedding:** Each chunk is embedded one at a time (not parallel) to respect Gemini's free-tier rate limits.

Function 2: assist — Answer with RAG

```
if (contentId && RETRIEVAL_TASKS.has(taskType) && userMessage) {
  const queryEmbedding = await embedText(userMessage)
  const { data: matches } = await supabase.rpc('match_content_chunks', {
    p_content_id: contentId, query_embedding: queryEmbedding, match_count: 6,
  })
  contentText = matches.map((m, i) => `[$${i+1}] ${m.chunk_text}`).join('\n\n')
  citations = matches.map((m, i) => ({ ref: i+1, snippet: m.chunk_text.slice(0,180) }))
}
```

`RETRIEVAL_TASKS` is a Set containing `'doubt'`. Only doubt questions use semantic retrieval — other tasks (summarise, simplify) use the full indexed text because they need to cover everything, not just the most relevant passages.

The chunks are numbered [1], [2], ... so Gemini can reference them inline. The `citations` array is returned to the frontend to display a "Grounded in this material" section showing which passages the answer came from.

Interview Questions — RAG Pipeline

Q: Walk me through exactly what happens when a student asks a doubt question.

A: (1) Frontend sends the question text and content ID to `POST /api/ai/assist`. (2) Server embeds the question using Gemini text-embedding-004 — converts it to 768 numbers. (3) Calls `match_content_chunks()` PostgreSQL function, which computes cosine similarity between the question vector and every stored chunk vector for that content. Returns top 6 most similar chunks. (4) Those 6 chunks are formatted as numbered passages [1], [2]... and passed to Gemini along with a system prompt saying "answer only from these passages and cite which ones." (5) Gemini returns an answer with inline citations. (6) Server sends back both the answer text and a citations array (ref number + snippet). (7) Frontend displays the answer and shows the citations panel.

Q: Why do you delete old chunks before re-ingesting?

A: Idempotency. If a creator updates their content and triggers re-ingestion, we don't want old and new chunks mixed together. Deleting first and re-inserting makes the operation idempotent —

running it multiple times always produces the same clean result: only the current version's chunks exist in the database.

CLIENT: supabase.js · authStore · themeStore

client/src/lib/supabase.js

```
export const supabase = createClient(import.meta.env.VITE_SUPABASE_URL,
import.meta.env.VITE_SUPABASE_ANON_KEY)
```

Creates one shared Supabase client for all frontend code. `import.meta.env.VITE_*` is how Vite exposes env vars to browser code. Only variables prefixed with `VITE_` are included in the browser bundle — this prevents accidentally exposing server secrets. The **anon key** is safe here because RLS policies control what it can do.

client/src/store/authStore.js – Zustand global state

Problem Zustand solves: The logged-in user's info (user object, profile with token balance) is needed in 10+ components: Navbar, ContentPage, Wallet, Upload, Dashboard, etc. Without global state, you'd have to pass user as a prop from App → Navbar → every child — called "prop drilling." Zustand creates a store any component can directly subscribe to.

`create((set) => ({...}))` — creates the store. `set` is the function to update state.
`setUser: (user) => set({ user })` is a function that takes a user object and updates the store. Any component calling `useAuthStore()` re-renders when the store changes.

Why not Redux? Redux requires actions, reducers, dispatch, selectors — 50+ lines of boilerplate for the same result. Zustand does it in 10 lines. **Why not React Context?** Context re-renders EVERY consumer when ANY value changes. If `token_balance` updates, Zustand only re-renders components that specifically subscribed to that field.

client/src/store/themeStore.js – Dark mode with persistence

`persist()` **middleware** wraps the store and automatically syncs it to `localStorage` under the key `'bb-theme'`. On page reload, Zustand reads `localStorage` and initialises the store with the saved value. Without `persist`, dark mode would reset to default on every page refresh.

`document.documentElement.classList.toggle('dark', next)` — adds or removes the `dark` class on the `<html>` element. Tailwind CSS checks for this class to switch colour schemes — `dark:bg-gray-900` only applies when the html element has the `dark` class.

```
client/src/components/common/ProtectedRoute.jsx
```

```
if (!user) return <Navigate to="/login" state={{ from: location }} replace />  
if (adminOnly && profile?.role !== 'admin') return <Navigate to="/" replace />
```

Navigate is React Router's component for programmatic redirects. When rendered, it immediately navigates the user to the specified route. `state={{ from: location }}` passes the current URL as state — the Login page reads this to redirect back after successful login. **replace** replaces the history entry instead of adding one, so pressing Back doesn't loop you back to the login page.

The admin guard checks `profile.role`. Why check after the user check? Profile is loaded asynchronously after the user session is restored. For a split second, user might exist but profile might still be null. We show a loading spinner instead of redirecting incorrectly.

```
client/src/App.jsx — Router setup and session restoration
```

BrowserRouter uses the HTML5 History API — URLs look like `/browse`, `/wallet` instead of `/#/browse`.

Routes is the container; each **Route** maps a URL pattern to a component. React Router compares the current URL and renders the matching component.

Session restoration in useEffect: When the page loads, we call `supabase.auth.getSession()` which checks if there's a stored JWT in `localStorage`. If yes, the user is still logged in — we fetch their profile and update the Zustand store. This is what keeps users logged in across page refreshes.

The `[]` dependency array in `useEffect` means "run this only once, on mount." Without it, it would run on every render — infinite loop.

The 4-step upload sequence

- ▶ **Step 1 — Upload file to Supabase Storage:**
`supabase.storage.from('content').upload(fileName, file)` — uploads the actual file bytes to S3-compatible storage. Returns a public URL. The `fileName` uses `user.id/timestamp.ext` format to avoid name collisions.
- ▶ **Step 2 — Upload thumbnail** (optional, same bucket): Only runs if the user selected a thumbnail image.
- ▶ **Step 3 — Insert database row:** `supabase.from('content').insert({...})` with all metadata. `is_published: false` means it starts as a draft — creator must explicitly publish from their dashboard.
- ▶ **Step 4 — Fire-and-forget AI ingest:** After the DB insert succeeds and we have the content ID, we call `/api/ai/ingest` WITHOUT await. The page shows success immediately; indexing happens in the background on the server.

Why fire-and-forget for AI ingestion?

Embedding a 30-page PDF requires downloading it, parsing text, splitting into ~50 chunks, and making 50 API calls to Gemini. This could take 10-20 seconds. If we awaited it, the user would sit on a loading screen for 20 seconds on every upload. Fire-and-forget lets the upload complete instantly; AI features become available after indexing finishes in the background. The `.catch(() => {})` ensures any indexing error doesn't bubble up and crash the upload success flow.

The autocomplete pattern

`TOPIC_SUGGESTIONS.filter(t => t.toLowerCase().includes(topicInput.toLowerCase()))` — filters the hardcoded list to suggestions containing the user's input. Shows up to 6 results.

The 150ms setTimeout on blur: `onBlur={() => setTimeout(() => setShowSuggestions(false), 150)}` — without this delay, clicking a suggestion causes: blur fires → dropdown hides → click registers on nothing. The 150ms delay lets the click on a suggestion register before the dropdown hides.

Interview Questions — Upload

Q: Why do you start AI indexing after upload completes instead of during?

A: Two reasons: (1) Performance — embedding a PDF requires downloading it, parsing text, and making potentially 50+ sequential API calls to Gemini. This takes 10-30 seconds. If we awaited it, the upload UX would feel broken. (2) Resilience — if indexing fails, the content still exists and is usable. We can retry indexing separately. The upload itself is fast (just a file transfer) and shouldn't be blocked by downstream AI processing.

Q: How does the topic autocomplete work?

A: It's entirely client-side — no API call. We have an array of 22 common topic suggestions. As the user types, we filter this array to only items that include the typed string (case-insensitive). Results are shown in a dropdown. When a suggestion is clicked, we set the input value to that suggestion.

The `onBlur` handler has a 150ms `setTimeout` before hiding the dropdown — this gives time for a suggestion click to register before the dropdown disappears.

Parallel data fetching in useEffect

Four Supabase queries run simultaneously (not sequentially with `await`). None of them block each other. React doesn't care which finishes first — each `.then()` updates its own state independently. This is faster than sequential fetching because the network round-trips overlap.

`.maybeSingle()` vs `.single()` — `.single()` throws a JavaScript error if zero rows are returned. `.maybeSingle()` returns `null` if no row exists. For unlock/bookmark/rating checks, zero rows means "user hasn't done this yet" — that's expected and not an error. Using `.single()` there would crash the page on every first visit.

handleUnlock — atomic token transfer

```
await supabase.rpc('unlock_content', {
  p_user_id: user.id, p_content_id: content.id,
  p_cost: content.token_cost, p_creator_id: content.creator.id,
})
```

This single RPC call does 4 things atomically in the database: (1) verify buyer has enough tokens, (2) deduct tokens from buyer, (3) credit tokens to creator, (4) insert an unlock record. If any step fails, all steps roll back — you can't end up in a state where tokens were deducted but content wasn't unlocked.

handleRating — self-rating prevention

`rate_content()` SECURITY DEFINER function checks `rater_id ≠ creator_id` at the database level. The frontend also checks to control the UI (showing the "+2 tokens" badge only for non-self ratings), but the database is the authoritative enforcement. A malicious user who bypasses the UI and calls Supabase directly cannot rate their own content.

AI Citations display

After `runAI()` for doubt questions, `data.citations` contains an array of `{ref, snippet, similarity}` objects. These are displayed below the AI response as "Grounded in this material" — showing the actual PDF passages the answer was derived from. This makes the AI trustworthy and verifiable — students can see exactly which part of their document was used.

Interview Questions — ContentPage

Q: Why do you use an RPC for unlocking content instead of directly updating the token balance?

A: Three reasons: (1) Atomicity — the unlock involves multiple operations: deduct from buyer, credit to creator, insert unlock record. A direct multi-query approach could leave the DB in an inconsistent state if the server crashes midway. The RPC wraps everything in one database transaction that either all succeeds or all rolls back. (2) Security — RLS blocks clients from directly updating `token_balance`. The SECURITY DEFINER function can bypass RLS safely for this specific audited operation. (3) Concurrency — if two requests come simultaneously, the database transaction handles locking, preventing race conditions.

Q: How do you prevent someone from rating their own content to earn tokens?

A: At two levels. The frontend checks `content.creator.id === user.id` and doesn't show the +2 token badge (and the `rate_content` function won't award tokens for self-rating). But the critical protection is at the database level — the `rate_content()` SECURITY DEFINER function has a check: if `rater_id = creator_id`, it still records the rating but skips the +2 token credit. Someone who bypasses our UI and calls Supabase directly with a crafted request still cannot earn tokens for self-rating.

ExamMode · Wallet · FeedbackButton

client/src/pages/ExamMode.jsx – AI study tools

5 tabs: Expected Questions, Mock Test, Revision Notes, Final Prep, Study Planner. Each maps to a different `taskType` sent to `/api/ai/exam`. The server maps these to system prompts in `gemini.js`.

Study Planner is different: It additionally sends a `days` parameter (3/5/7/14/21/30). The server includes "Study duration: N days" in the context, and the special study-plan system prompt tells Gemini to create a day-by-day progressive plan. This is "agentic" — it plans across time rather than answering a single question.

Free-text inputs instead of dropdowns: Subject and topic are now plain text inputs so any subject works — not just the 4 hardcoded ones from the old version. This makes Exam Mode usable for any student with any curriculum.

client/src/pages/Wallet.jsx – Token purchase and withdrawal

Buy tokens flow: (1) POST `/api/payment/create-order` with INR amount → get `orderId`. (2) Open Razorpay checkout with `orderId` — user pays on Razorpay's interface. (3) On success callback, POST `/api/payment/verify` with `order_id`, `payment_id`, `signature`. (4) Server verifies HMAC and credits tokens. (5) Update local profile balance.

Withdrawal: `supabase.rpc('request_withdrawal', { p_tokens, p_method, p_upi, p_bank_name, p_account, p_ifsc })` — this SECURITY DEFINER function checks balance, deducts tokens, and creates a withdrawal request in one atomic operation. An admin must approve it before money is transferred.

Why use RPC for withdrawal? Same reason as unlock — atomicity. Checking balance, deducting tokens, and creating the withdrawal record must all happen together. If they were separate queries, a race condition could let two simultaneous withdrawals both pass the balance check before either deducts.

`fixed bottom-6 right-6 z-40` (Tailwind) — CSS position:fixed means the button stays in the same viewport position regardless of scrolling. bottom-6 = 24px from bottom, right-6 = 24px from right. z-40 ensures it appears above most page content.

Anonymous feedback: `user_id: user?.id || null` — if not logged in, user_id is null. The database accepts null here. RLS allows anyone (including anonymous) to INSERT into the feedback table.

Auto-reset after submit:

`setTimeout(() => { setOpen(false); setSubmitted(false); setForm({...}) }, 2000)` — shows success state for 2 seconds, then resets everything. Clean UX without requiring the user to manually close the modal.

Key Design Decisions

Row-Level Security (RLS)

Every table has RLS enabled. This means: even if someone gets hold of the anon key and calls Supabase directly (bypassing our Express API), the database itself enforces what they can do. The profiles table policy explicitly EXCLUDES token_balance and role from user updates: `WITH CHECK (token_balance = (SELECT token_balance FROM profiles WHERE id = auth.uid()) AND role = (SELECT role FROM profiles WHERE id = auth.uid()))` — users can update their name/avatar but the token balance and role must stay the same.

SECURITY DEFINER Functions

These are PostgreSQL functions that run with the privileges of the function owner (who has elevated access), not the caller (who has limited access). Our six SECURITY DEFINER functions: `credit_purchase()`, `request_withdrawal()`, `rate_content()`, `refund_tokens()`, `increment_views()`, `match_content_chunks()`. Each is a specific, audited operation that needs to bypass RLS for a good reason — none of them allow arbitrary SQL execution.

pgvector Setup

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE content_chunks (
  id          uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  content_id  uuid REFERENCES content(id) ON DELETE CASCADE,
  chunk_index integer,
  chunk_text  text,
  embedding   vector(768) -- 768 floats from Gemini text-embedding-004
);

CREATE INDEX ON content_chunks
  USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```

`vector(768)` — custom data type from pgvector extension. Stores 768 floating-point numbers per row.

IVFFlat index — Inverted File Flat index. For similarity search over thousands of vectors, checking every vector (brute force) is slow. IVFFlat groups vectors into 100 clusters. When searching, it only checks the most likely clusters — much faster approximate nearest-neighbour search. `vector_cosine_ops` tells it to use cosine similarity as the distance metric.

ON DELETE CASCADE — if a content row is deleted, all its chunks are automatically deleted too. No orphaned chunks.

match_content_chunks() function

```
CREATE FUNCTION match_content_chunks(p_content_id uuid, query_embedding vector(768),
match_count int)
RETURNS TABLE(chunk_text text, similarity float)
LANGUAGE sql SECURITY DEFINER AS $$
    SELECT chunk_text, 1 - (embedding <=> query_embedding) AS similarity
    FROM content_chunks
    WHERE content_id = p_content_id
    ORDER BY embedding <=> query_embedding
    LIMIT match_count;
$$;
```

`<=>` is the pgvector cosine distance operator. Lower distance = more similar. `1 - distance = similarity`. So we order by distance (ascending = most similar first) and return `1 - distance` as the similarity score.

Interview Questions — Database

Q: How does your database prevent users from directly manipulating their token balance?

A: Row-Level Security. The profiles table has an UPDATE policy that allows users to update their own row BUT requires that token_balance and role remain unchanged. The policy uses a WITH CHECK clause that compares the proposed values against the current values — if they differ, the update is rejected by the database engine itself, regardless of how the request was made. Even a developer with the anon key calling Supabase directly cannot bypass this.

Q: What is pgvector and why did you use it instead of a dedicated vector database?

A: pgvector is a PostgreSQL extension that adds a vector data type and similarity search capabilities including IVFFlat indexing for approximate nearest-neighbour search. I already use Supabase (PostgreSQL) for all my data — adding pgvector keeps everything in one database. Dedicated vector databases like Pinecone or Weaviate would require a separate service, separate credentials, separate billing, and data synchronisation between systems. For the scale of this project, pgvector in PostgreSQL is more than sufficient and dramatically simpler to operate.

Architecture & Design

1. Why did you split the app into a separate React frontend and Express backend instead of using Next.js?

A: Several reasons. First, our backend does heavy work — PDF parsing, calling Gemini 50 times per ingestion, HMAC verification — that benefits from a persistent Node.js process on Render rather than serverless functions. Second, separating frontend and backend means they can scale independently. Third, having a standalone API means it can serve mobile apps or other clients in the future. Next.js was designed for SSR-first apps; ours is a SPA with a complex backend.

2. Why Supabase instead of Firebase?

A: Firebase uses a NoSQL document store — no SQL queries, no JOINS, no pgvector extension. Our app needs: (1) Complex relational queries — content joined with creators, unlocks, ratings. (2) pgvector for semantic search. (3) Row-Level Security for fine-grained access control. Firebase can't do any of these. Supabase gives us PostgreSQL with all its power plus Auth and Storage in one platform.

3. How does the token economy work at a technical level?

A: All token movements are handled by SECURITY DEFINER PostgreSQL functions: `credit_purchase()` for buying, `unlock_content()` for spending, `rate_content()` for earning +2, `request_withdrawal()` for cashing out, `refund_tokens()` for admin rejections. None of these can be called directly by clients with an anon key — they go through our Express API (which verifies the JWT first) or are triggered by server-side logic. The profiles table RLS prevents any direct manipulation of `token_balance`.

4. How does your app handle concurrent requests — e.g. two users unlocking the same content at the same time?

A: The `unlock_content()` database function handles this. PostgreSQL transactions with appropriate isolation levels ensure that two concurrent unlock operations for the same user are serialised at the database level. The function checks balance and deducts atomically — a `SELECT FOR UPDATE` lock prevents another transaction from reading and modifying the same balance row simultaneously.

5. What would you change if this needed to scale to 100,000 users?

A: (1) The embedding ingestion is currently sequential — I'd add a message queue (Redis + Bull) to process uploads in parallel workers. (2) The IVFFlat index with 100 lists would need more lists as the vector table grows. (3) CDN for video delivery instead of direct Supabase Storage URLs. (4) Caching frequent queries (leaderboard, popular content) with Redis. (5) Horizontal scaling of the Express API behind a load balancer.

Security

6. How does HMAC payment verification work and why can't it be faked?

A: HMAC uses a secret key to produce a message authentication code. Our server and Razorpay both know the same secret (`RAZORPAY_KEY_SECRET`). When a payment completes, Razorpay computes `HMAC-SHA256(order_id + "|" + payment_id, secret)` and sends it as `razorpay_signature`. Our server does the same computation. If they match, we know the message hasn't been tampered with AND it came

from Razorpay. An attacker cannot forge a valid signature without knowing the secret key — SHA256 is a one-way function, you can't reverse it to find the key.

7. What is the difference between authentication and authorisation in your app?

A: Authentication = "who are you?" — handled by Supabase Auth + JWT. The `verifyToken` middleware authenticates every request. Authorisation = "what are you allowed to do?" — handled by RLS policies and explicit checks. For example, `ingestContent` authorises by checking `creator_id === req.user.id`. The admin panel authorises by checking `profile.role === 'admin'`. Authentication happens at the middleware layer; authorisation happens in each specific handler or RLS policy.

8. Can a user with the anon key directly query Supabase and steal all content?

A: No. RLS policies on the content table only allow reading content that is published (`is_published = true`). Unpublished drafts are invisible. The `content_chunks` table has RLS policies that prevent reading chunks directly — only the `match_content_chunks()` SECURITY DEFINER function can query them, and it's called by our Express API (not exposed directly). Actual file URLs from Supabase Storage use signed or public URLs configured to be accessible only for content the user has unlocked.

9. How are admin permissions enforced?

A: At two levels. Frontend: `ProtectedRoute` with `adminOnly` prop checks `profile.role` — non-admins see a redirect to /. Backend: admin-only operations go through our Express API which could add role checks (or they rely on RLS). The role is stored in the profiles table with an RLS policy preventing users from changing their own role — only a service-role client (our server) can change roles.

10. What prevents someone from sending fake feedback or reports spam?

A: The feedback and reports tables require INSERT — anyone can submit. Rate limiting would be the proper solution (not yet implemented). However, the admin panel gives moderators full control to review and dismiss reports. The `user_id` (or null for anonymous) is recorded, so patterns can be identified. For production at scale, I'd add rate limiting middleware (`express-rate-limit`) and CAPTCHA for anonymous submissions.

AI & RAG

11. How is your RAG implementation different from just passing the whole PDF to Gemini?

A: Three important differences. (1) Scale — a 50-page PDF might have 25,000 words, exceeding Gemini's context window. RAG lets us handle documents of any size. (2) Cost — Gemini charges per token. Sending 25,000 words every time someone asks a question is expensive. RAG sends only the 6 most relevant passages (~1,320 words). (3) Precision — flooding the context with irrelevant text actually degrades answer quality. Retrieval selects only relevant passages, grounding the answer more precisely.

12. What model do you use for embeddings vs generation and why are they different?

A: `text-embedding-004` for embeddings — this model is specialised for converting text to vectors that capture semantic meaning. It doesn't generate text; it just maps text to a 768-dimensional space. `gemini-1.5-flash` for generation — this is the conversational/generative model that produces human-readable answers, summaries, study plans, etc. They're separate models because they do fundamentally different things. Both use the same Gemini API key — no extra cost for embeddings on the free tier.

13. What happens if a student asks a question the PDF doesn't cover?

A: The system prompt for doubts explicitly says "If the passages do not contain the answer, say so honestly." So Gemini will retrieve the closest available chunks (which might be tangentially related) and if they don't contain the answer, it will say something like "The provided material doesn't cover this

specific question." It won't hallucinate an answer from general training data. This honesty is a feature — students know to look elsewhere when the document doesn't cover something.

14. How does the Study Planner differ from a regular AI chat?

A: A regular chat answers a single question in isolation. The Study Planner is "agentic" because it plans across time — given a subject and N days, Gemini produces a structured sequence of daily goals, each building on the previous. The system prompt instructs it to be progressive (foundations first, then advanced concepts, then revision). The user gets a cohesive multi-day plan rather than a single answer. This is a different kind of AI usage — planning rather than answering.

Frontend & React

15. Why did you choose Zustand over Redux or Context API?

A: Redux requires actions, action creators, reducers, and dispatch — 50+ lines of boilerplate to store whether a user is logged in. Zustand achieves the same in 10 lines. React Context re-renders every component that consumes it whenever any value in the context changes — if `profile.token_balance` updates, every component using the auth context would re-render, even if they don't care about the balance. Zustand uses subscriptions — components only re-render when the specific slice of state they use changes. For a project of this size, Zustand was the perfect balance of simplicity and performance.

16. How does dark mode persist across page refreshes?

A: The `themeStore` uses Zustand's `persist` middleware, which serialises the store to `localStorage` under a key (`'bb-theme'`). On page load, the middleware reads `localStorage` and initialises the store with the saved value. The `init()` function then applies the class to the HTML element:
`document.documentElement.classList.toggle('dark', savedDarkValue)`. Tailwind CSS reads this class to switch between light and dark colour schemes. The whole thing is transparent — the user just sees their preference remembered.

17. What is the difference between `useEffect` with `[]` and without dependencies?

A: `useEffect` without a dependency array runs after EVERY render — every state change, every prop change. `useEffect` with an empty array `[]` runs ONLY once, after the initial render (like `componentDidMount`). `useEffect` with `[id, user]` runs after the initial render AND whenever `id` or `user` changes. In `ContentPage`, we use `[id, user]` because when the URL changes to a different content page, we need to re-fetch data for the new content ID.

18. How does `ProtectedRoute` redirect back to the original page after login?

A: When `ProtectedRoute` redirects to `/login`, it passes the current location as state: `<Navigate to="/login" state={{ from: location }} replace />`. The Login page reads this: `const { state } = useLocation()`; and after successful login, navigates to `state?.from?.pathname || '/'` — the original URL the user was trying to access. The `replace` prop prevents the `/login` entry from appearing in the browser history, so pressing Back after login goes to the page before the protected route.

General / Quick Fire

19. What is the difference between `.env` variables prefixed with `VITE_` and those without?

A: Vite (our frontend build tool) only includes env variables prefixed with `VITE_` in the browser bundle. Variables without this prefix are never sent to the browser. This prevents accidentally exposing server

secrets like `RAZORPAY_KEY_SECRET` or `SUPABASE_SERVICE_KEY` in the frontend JavaScript that anyone can read in browser DevTools.

20. What does `async/await` do and why do we use it?

A: `async/await` is syntactic sugar for Promises — JavaScript's way of handling asynchronous operations (network requests, file reads, database queries) without blocking. Without it, you'd write `.then().then().catch()` chains that are hard to read. With `async/await`, the code reads linearly like synchronous code. `await` pauses the function until the Promise resolves. The `async` keyword marks a function as containing `await` and automatically returns a Promise.

21. What is the purpose of the receipt field in the Razorpay order?

A: It's a unique identifier we generate for our own records — ``bb_${Date.now()}`` produces something like `"bb_1703123456789"`. It doesn't affect payment processing but helps us identify orders in the Razorpay dashboard and in our logs. Using `Date.now()` (current timestamp in ms) ensures uniqueness for all practical purposes.

22. What is the purpose of the health check endpoint?

A: Render (our hosting platform) periodically pings `/health` to confirm the server is running. If the endpoint stops responding, Render restarts the server. External monitoring services (UptimeRobot, etc.) also use health checks. It's a one-liner that returns 200 OK — minimal overhead but critical for uptime monitoring and automated recovery.

23. Why is `is_published` set to false on upload?

A: Safety. A creator might upload content that needs editing before it's public. By defaulting to draft (`is_published: false`), they can review it in their dashboard, make changes, and only publish when ready. This prevents accidentally publishing incomplete or incorrect content the moment it's uploaded.

24. What is the purpose of the `onConflict: 'email'` in the `upsert` call?

A: `upsert` means "insert if not exists, update if exists." The `onConflict` tells Supabase which column to check for conflicts. If a row with the same email already exists (user is re-requesting a verification code), it updates that row with the new code and expiry instead of trying to insert a duplicate and getting a unique constraint error. This handles the "resend code" flow gracefully.

25. How does the progress bar on the Upload page work?

A: It's a manually controlled state value — not actually measuring real upload progress from the network. We use `setProgress(30)` before the file upload, `setProgress(60)` after file upload completes, `setProgress(80)` after thumbnail upload, `setProgress(100)` after DB insert. The bar visually fills in stages corresponding to these steps. True byte-level progress would require monitoring the upload XHR, which is more complex and wasn't necessary here.

"**BrainBarter is a peer-to-peer learning marketplace** I built where students earn tokens by uploading study material and spend those tokens to unlock content from peers. Creators can withdraw their earnings as real money to UPI or a bank account.

The interesting technical part is the **AI tutor**. Instead of just using ChatGPT-style general knowledge, I built a RAG pipeline — Retrieval-Augmented Generation. When a student uploads a PDF, my server extracts the text, splits it into 220-word overlapping chunks, converts each chunk to a 768-dimensional vector using Google Gemini's embedding model, and stores those vectors in PostgreSQL using the pgvector extension. When a student asks a question, we embed the question the same way, run a cosine similarity search to find the most relevant passages, and send only those passages to Gemini to answer from — with citations to the exact passages used.

For **security**, I was careful about three things: payment integrity — Razorpay payments are verified server-side using HMAC-SHA256 so no one can fake a payment. Token safety — all token movements go through PostgreSQL SECURITY DEFINER functions, and Row-Level Security on the profiles table prevents any direct manipulation of balances. Authentication — email OTPs use Node's crypto.randomInt instead of Math.random, stored in Supabase so they survive server restarts.

The stack is **React with Zustand** for global state on the frontend, **Node.js with Express** as the API, **Supabase** for PostgreSQL with Auth and Storage, and deployed on **Vercel and Render**."

Tips for the interview

- ▶ When asked about **any security decision**, always mention the layer — client-side check (UX), server-side check (API), and database-level enforcement (RLS/SECURITY DEFINER). Show you think in defence-in-depth.
- ▶ When asked "**why X over Y?**" — always give a concrete reason tied to your project's needs, not just "X is better."
- ▶ When asked about **RAG** — draw the two-phase diagram: ingestion (upload → chunk → embed → store) and retrieval (question → embed → similarity search → feed to AI).
- ▶ When asked about **HMAC** — say "both sides have the same secret, both compute the same hash, if they match the message is authentic and unmodified."
- ▶ When asked about **what you'd improve** — mention: rate limiting, proper signed URLs for content access, Redis caching for the leaderboard, job queue for bulk embedding.